第2章　C++对C的扩展

问：C++是面向对象化的？而C是面向过程化的？

答：C++并非完全面向对象化，真正的面向对象化的语言Java。

问：学了C再学C++有障碍吗？比如程序设计思想

答：初期会觉得“类”是一个累赘。类的封装会使得初学者对程序产生厌倦，感到不适和麻烦。

问：学了C再学C++又要重头开始吗？

答：不，C++下可以实现C语言的一切功能。

问：C++能实现C所不能的功能吗？

答：至少我还没有发现

问：我学完了C一定还要学C++才能编程吗？

答：完全没必要。

问：C++比C好在哪里？

答：更加符合软件工程学

问：学完了C再学C++是不是很容易？

答：那要看你是不是真正的学会了C语言。

第二章 C++对C的扩展

1. C++的特点
2. C++的源程序名
3. C++注释行
4. 命名空间
5. 新的I/O
6. 结构、联合和枚举名
7. 常量——const修饰符
8. 函数原型
9. 内联函数
10. 带有默认参数的函数
11. 重载函数
12. 作用域运算符 “::”
13. 强制类型转换
14. new 和 delete运算符
15. 引用

1. C++的特点

(1) C++保持与C兼容，许多C代码不经修改就可以为C++所用，用C编写的众多的库函数和实用软件可以用于C++中。

(2) 用C++编写的程序可读性更好，代码结构更为合理，可直接在程序中映射问题空间的结构。

(3)生成代码的质量高，运行效率仅比汇编语言代码段慢10%到20%。

(4)从开发时间、费用到形成的软件的可重用性、可扩充性、可维护性和可靠性等方面有了很大的提高，使得大中型的程序开发变得更加容易。

(5)支持面向对象的机制，可方便地构造出模拟现实问题的实体和操作。

6

1. C++的特点

总之，目前人们对C++的兴趣越来越浓，它已经成为被广泛使用的通用程序设计语言。在国内外使用和研究C++的人正迅猛增加，优秀的C++版本和配套的工具软件不断涌现。

2. C++的源程序名

- C源程序文件扩展名为.c,而C++源程序文件扩展名为.cpp。

```
// sum.cpp
#include<iostream.h>
int add(int a, int b);                  //函数原型的说明
int main()                              //主函数
{
 int x, y, sum;                         //定义三个整型变量
 cout <<"Enter two numbers:"<<'\n';     //界面,提示用户输入两个数
 cin>>x;                                //从键盘输入变量 x 的值
 cin>>y;                                //从键盘输入变量 y 的值
 sum=add(x, y);                         //调用函数 add,将得到的值赋给变量
                                        //sum
 cout<<"The sum is:"<<sum<<'\n';        //输出两个数的和 sum 的值
 return  0;
}
```

```
int add(int a,int b)                    //定义 add 函数,函数值为整型
{
 int c;                                 //定义一个整型变量
 c=a+b;                                 //计算两个数的和
 return c;                              //将 c 的值返回,通过 add 带回调用处
}
```


3. C++注释行

C++是从C发展而来，C程序中的表达式、语句、函数和程序的组织方法等在C++中仍可以使用。

- 在C语言中用“/*”及“*/”作为注释分界符号,C++除保留了这种注释方式外，还提供了一种更有效的注释方式，该注释以“//”开始，到行尾结束。例如以下两条语句是等价的：

C: x=y+z ; /* This is a comment*/

C++: x=y+z; // This is a comment

4. 命名空间

- namespace 更加灵活的文件域，可以用花括号把文件的一部分括起来

```
namespace ns1{
    float a, b, c;
    void fun(){}
}
```

调用： ns1::a,ns1::fun();

- namespace可以嵌套

```
namespace ns1{
    namespace ns2{
        float x;
    }
}
```

调用: ns1::ns2::x;

4. 命名空间

- 避免每次访问都用namespace
  - -using namespace + 命名空间
  - -using + 变量

定义：

```
namespace ns1{
    float a, b, c;
    void fun(){}
}
```

调用访问：（1）ns1::a=0.5f; ns1::b=0.6f;

（2）using ns1::a;

a=0.5f; (√) b=0.6f; (╳) ns1::b=0.6f; (√)

（3）using namespace ns1;

a=0.5f; (√) b=0.6f; (√)

4. 命名空间

- C++标注库中的函数在std命名空间中

（1） #include <iostream.h>

cout<<“test1”<<endl;

cout<<“test2\n”;

（2）#include<iostream>

std::cout<<“test1”<<std::endl;

std::cout<<“test2\n”;

（3）#include<iostream>

using namespace std;

cout<<“test1”<<endl;

cout<<“test2\n”;

std::cout<<“test”<<endl; (?)

5. 新的I/O

- 在C中进行I/O操作时，常会出现下面的错误：

```
int a;
float f;
scanf(" %f ", a); // ×
printf(" %d ", f); // ×
```

%d　&a　%f

- scanf()和printf()所使用的格式控制符与输出数据的类型不一致
- scanf()的第二个参数应是一个指针参数

- C++使用了更安全和更方便的方法来进行I/O操作，上面的程序段可以使用C++的I/O操作写为：

```
int a;
float f;
cin>>a;
cout<<f;
```

- 这里的cin是标准的输入流，在程序中用于代表标准输入设备，即键盘。

5. 新的I/O

◈ cin>>x>>y;

3　4 — cin — >> — x=3; y=4;

◈ cout << "This is my first program! ";

cout — << — "This is my first program！"

不管把什么基本数据类型的名字或值传给流，它都能懂。

5. 新的I/O

■ 屏幕显示输出：用插入操作符"<<"向cout输出流中插入字符：

cout<<"This is a program.\n"

例：

```
# include<iostream.h>
int main()
{
  cout<<"My name is Peter\n";
  cout<<"the ID is ";
  cout<<2;
  cout<<endl;
}
```

等价于：

```
cout<<"My name is Peter\n"
    <<"the ID is "
    <<2
    <<endl;
```

```
cout<<"My name is Peter\n"<<"the ID is "<<2<<endl;
```


5. 新的I/O

■ 键盘输入：使用抽取操作符">>"从cin输入流中抽取字符：

```
int myAge;
cin>>myAge;

int i; float f; long l;
cin>>i>>f>>l;
```


6. 结构、联合和枚举名

结构、联合和枚举名可直接作为类型名

```
enum bool{FALSE,TRUE};
struct string{
  char * str;
  int length;
};
```

在定义变量时，可以说明为：

```
bool done;
string str;
```

但是在传统的C语言中，必须写成：

```
enum bool done;
```

7. 常量——const修饰符

 # define

✓是c的编译预定义命令，c++中用*const*而不用 *# define*

✓#define 不是C++的语句，C++允许#define是为了兼容c

✓语法：

后面无分号

 # define PI 3.1415926

const

常数或固定不变值的名字

const 类型名 常量名；
const float pi=3.14159;

✓常量必须赋初值，且程序对其只能读不能修改，常量不能放在赋值语句左边

const float pi=3.14159;
pi=3.142593; *//error*
const float area;
area=9.43; *//error*

7. 常量——const修饰符

◈ const的作用与#define相似，但它消除了#define的不安全性

```
#include<iostream.h>
main()
{
 int a=1;
 #define  T1  a+a;
 #define  T2  T1-T1;
 cout <<"T2 is"<<T2<<endl;
 return 0;
}
```

初看程序，似乎应打印出：
T2 is 0
但是这个答案是错误的，程序的实际输出结果是：
T2 is 2
其原因是C++把第7行语句解释成：
cout<<"T2 is"<<a+a-a+a<<endl;

```
#include<iostream.h>
int main()
{
  int a=1;
  const T1=a+a;
  const T2=T1-T1;
  cout <<"T2 is"<<T2<<endl;
  return 0;
}
```

输出：T2 is 0

7. 常量——const修饰符

◈ 注意事项

- #define 定义的是常量，没有数据类型，const定义的变量有自己的数据类型，C++编译器可以更严格检查
- const定义变量常量时必须初始化，一旦定义不能再改
- const定义int类型常量，int可以省略
- 函数的参数可以用const说明，而且经常用，如果不希望函数对参数修改，这样编译器可以对函数进行优化。如，通过函数max求整数数组a[100]中最大值

```
int max(const int* a);
int max(int const* a);(?)
int max(int * const a);(?)
```


7. 常量——const修饰符

用const限定指针

使用限定词const限定变量可以使变量的值为一常量。同样地，限定词const也可以用来限定指针变量。根据const关键字在指针变量声明中出现位置的不同，可以得到多种const指针类型：

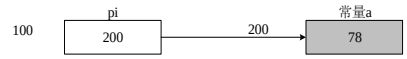
✓**指向常量的指针：** 指针把它指向的对象视为常量
const int *p 或： int const *p

✓**指针常量（常指针）：** 指针本身为常量，指针指向的地址不能改变： **int * const p**

✓**指向常量的指针常量：** 指针本身为常量，指针把它指向的对象也视为常量： **const int * const p**

7. 常量——const修饰符

指向常量的指针： const int *pi 或 int const *pi




```
const int a = 78;
const int b=  28;
int c = 18;
const int *pi = &a;
*pi = 58; // error
pi = &b; // ok
pi = &c; //ok
*pi=78; //error
c= 98;  //ok
```

✓定义指向常量的指针
- 指针指的值不能改
- 指针可以改（重新指向其他位置）

7. 常量——const修饰符

指针常量（常指针）： int * const pc

```
char str[5] = "asdf";
char *const pc =str;//必须初始化
pc = "dfgh"; // error
*pc = 'b'; // ok，内容为bsdf
*(pc+1) = 'c'; // ok,内容为bcdf
*pc++ = 'y'; // error
const int b = 28;
int *const pi = &b; // error
```

✓pc是常量，不能修改（不能指向其他位置）
✓pc所指的值可以修改，即*pc可以修改。

7. 常量——const修饰符

指向常量的指针常量： const int * const p

```
const int ci = 7;
int ai;
const int *const cpc = &ci;
                //必须初始化
const int *const cpi = &ai;
        //必须初始化,可以指向变量
cpc= &ai; // error
*cpi = 39; // error
ai = 39; // ok
```

cpc和*cpc都是常量，它们**都不能作为左值进行操作**。

注1： const总是修饰其前面内容，前面没有内容则修饰其后内容

注2： 平时使用时，尽量做到const修饰数据型变量，const放在前面；const修饰指针型变量，const放在后面

const int const* p; (?)

8. 函数原型

```
#include<iostream>
void write(char *s);
void main()
  {write("Hello,world!");}

void write(char *s)
  {cout<<s;}
```

（1）函数原型的参数表中可不包含参数的名字，而只包含它们的类型。例如以下的函数原型是完全合法的：

✓ `int Area(int , int );  //c++;c`

✓ `int  Area(int W, int L);  //c++;c`

`int  Area( );  //c,函数声明可以不带参数`

`Area( );  //c, 当返回值为int型时，可以省略int`

8. 函数原型

(2)如果函数原型中未注明参数，C++假定该函数的参数表为空(void)。

f(void) ; //c++、c都解释为无参函数

f( ); //c++：无参函数；c:可能带有参数，但参数没有给出

9. 内联函数

```
#include <iostream.h>
int isnumber(char);     //函数声明
int main()
{ char c;
  while((c=cin.get())!='\n') //从键盘输入的字符串中逐个读取字符并赋给c

   if( isnumber(c) )        //调用一个小函数，当条件不成立时
                            //反复被调用，反复进栈出栈
       cout <<"you entered a digit\n";
   else
       cout <<"you entered a non_digit\n";
}

int isnumber(char ch) //函数定义
{
  return (ch>='0' && ch<='9')? 1:0;
}
```

等价于： c>='0' && c<='9')? 1:0

9. 内联函数

```
#include <iostream.h>
inline int isnumber(char);   //函

void main() {
  char c;
  while((c=cin.get())!='\n') //从键
   if( isnumber(c) )          //调用
    cout <<"you entered a digit\n
   else
    cout <<"you entered a non_d
}

int isnumber(char ch)     //此处无
{
  return (ch>='0' && ch<='9')? 1:0;
}
```

✓把函数体直接嵌入到函数调用处。

✓编译时在调用处用函数体进行替换，节省了参数传递、控制转移等开销。

✓内联函数——提高效率　用关键字**inline**声明，函数定义处的inline可以省略

✓限制：

1、不含递归
2、不含静态数据
3、不含循环
4、不含switch、goto
5、不含数组

10. 带有默认参数的函数

C++在说明函数原型时，可为一个或多个参数指定缺省参数值，以后调用此函数时，若省略其中某一参数，C++自动地以缺省值作为相应参数的值。

例：

int special (int x＝5，float y＝5.3);

调用，

special(2，15.3);
special(23);
special( );

10. 带有默认参数的函数

几点说明：

(1)在函数原型中，所有取缺省值的参数都必须出现在不取缺省值的参数的右边。亦即，一旦开始定义取缺省值的参数，就不可以再说明非缺省的参数，例如：

int fun (int i，int j＝5，int k); 错误
int fun (int k，int i=8, int j＝5)　正确

(2)在函数调用时，若某个参数省略，则其后的参数皆应省略而采用缺省值。不允许某个参数省略后，再给其后的参数指定参数值。

fun(4, , 7); //错误
fun(4); //正确

11. 重载函数

- ✓ 在C++中，当两个或者两个以上的函数共用一个函数名时，称为函数重载。
- ✓ 被重载的函数称为重载函数。
- ✓ 将一组功能非常相近的函数定义为重载函数，它们是以参数类型或参数个数加以区别的，C++编译器依据传递实参类型和个数确定重载函数的调用。

➢在C语言中，不能同名

```
int abs(int);
long labs(long);
double fabs(double);
```

➢在C++中，可以同名

```
int abs(int);
long abs(long);
double abs(double);
```

11. 重载函数

✓**如何区分几个同名的重载函数**

用函数接口的两个要素之一——参数。如果同名的函数的参数不同（包含类型、个数和顺序的不同），那么容易区别。

void EatBeef(…); ————void Eat (Beef…);
void EatFish(…); ———— void Eat (Fish…);
void EatChicken(…); —— void Eat (Chicken…);

11. 重载函数

✓仅仅是返回类型不同是不能实现函数重载的；

```
int func(int);
void func(int);
func(3);  //调用哪个函数？
```

✓因此：**重载函数至少在参数个数、参数类型或参数顺序上有所不同。**

✓不能用typedef定义的类型来区分重载函数声明中的参数：
typedef int integer;

✓让重载执行不同功能，是不好的。

11. 重载函数

```
int abs(int val)
{
  return (val<0)?-val:val;
}
long abs(long val)
 {
  return (val<0)?-val:val;
 }
double abs(double val)
 {
   return (val<0)?-val:val;
 }
```

```
int main()
 { int i=100;
   cout<<abs(i)<<endl;
   long l=-12345L;
   cout<<abs(l)<<endl;
   double d=-125.78;
   cout<<abs(d)<<endl;
   float f=-125.78;
   cout<<abs(f)<<endl;
 }
```

12. 作用域运算符 "::"

这是C++所特有的，C语言中没有。
在C++语言中，可以通过(::)来标识引用同名的全局变量。

```
int s=0;
void f()
{  float s=3.0;  //和全局变量同名
   int a;
   {   float a=2.0;
       a=1;      //对，  访问float  a
      ::a=1;     //错，  a不是全局变量
      ::s=1;     //通过::来访问全局变量s
      s=2.0;     //访问局部变量s
   }
}
```

13. 强制类型转换

✓在c中如果要把一个整数转换为浮点数，要求使用如下的格式：

int i＝10 ; float x＝(float)i;

✓C++支持这样的格式，但还提供了一种更为方便的函数调用方法，即将类型名作为函数名使用，使得类型转换的执行看起来好像调用了一个函数。上面的语句可改写成：

int i＝10; float x＝float(i);

✓以上两种方法c++都能接受，但推荐使用后一种方式。

```
int i=5; int j=10; float m;
m=(float) (i+j);  //c/c++
m=float(i+j);  //c++
```

14. new 和 delete运算符

C/C++语言的内建数组属于静态数组，程序运行前就必须确定其数组容量。一旦定义了一个数组，那么不管这个数组是局部的还是全局的，它的大小在程序编译时即是已知的，因为必须用一个常数对数组的大小进行声明：

int i = 10; int a[i]; *//错,定义时不允许数组元素个数为变量*

const int i = 10; int a[i]; *//正确*

int b[20]; *//正确*

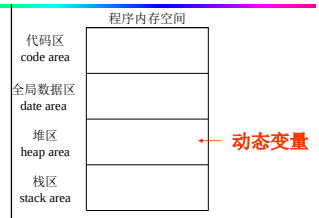
但是，有的数组需要的容量在程序运行后才会知道；对于这些数组，如果使用静态数组只能为其预先设定一个足够大的数组容量，因此造成内存空间的浪费。

14. new 和 delete运算符

✓堆是内存空间，堆允许程序在运行时（而不是在编译时），申请某个大小的内存空间。

✓如在编译和链接时不能确定内存空间大小，而是要随着程序运行的进展而确定内存空间大小。可以在程序运行过程中申请内存——**堆内存**。

✓堆内存就是动态内存。



两种方法：

1、用系统函数**malloc、free**

2、C++提供的运算符**new、delete。**

14. new 和 delete运算符

malloc和free函数

在malloc.h中声明

✓**malloc函数:**从堆上获得指定字节数的内存空间：

void * malloc(size_t size);

✓形参为要求分配的字节数。

✓如果函数执行成功，malloc函数返回获得的内存空间的首地址；如果函数执行失败，那么返回值为NULL。

✓由于malloc函数值的类型为void型指针，因此可以将其值类型转换后赋值给任意类型指针，这样就可以通过操作该类型指针来操作从堆上获得的内存空间。

14. new 和 delete运算符

free函数

✓从堆上获得的内存空间在程序结束后不会自动释放，需要程序员自己来管理。一个程序结束时，必须保证所有从堆上获得的空间已被安全释放，否则会导致内存泄露。

✓free函数可以实现释放内存的功能。其函数声明为：

void free(void * p);

✓形参为void指针，free函数可以接受任意类型的指针实参

✓free函数只释放指针指向的内容，而该指针的值仍然指向原来指向的地方，在使用free函数释放指针指向的空间之后，需要将指针值置为NULL：

free(p); *//释放堆内存*

p = NULL; *//指针值置成空*

14. new 和 delete运算符

```
int main()
{ int arraysize;    //元素个数
  int *array; //不能是：int array[];
  cout <<"please input a number of array:\n";
  cin >>arraysize;
  if((array=(int*)malloc(arraysize*sizeof(int)))==NULL)
   {  cout <<"can't allocate more memory, terminating.\n";
      exit(0); }
  for(int count=0; count<arraysize; count++)
    array[count]=count*2;
  for(count=0; count<arraysize; count++)
     cout <<array[count] <<"  ";
  cout <<endl;
  free(array);    //释放堆内存
  array=NULL;    //指针值置为空,注意NULL为大写
}
```

14. new 和 delete运算符

✓ new和delete是C++专有的运算符，不用头文件声明。

✓ new类似于函数malloc()，分配堆内存，但比malloc()更简练。

✓ delete类似于函数free()，释放堆内存。delete的操作数是new返回的指针。

new运算符:用于获得堆内存

指针变量 = new 数据类型[长度];

```
char *pBuffer = new char[256];
float *pNum = new float;            //分配空间长度为1
float *pData = new float(255.0);    //分配空间1，初值255.0
float *pData=new float[10](255.0); // 错误
```

✓动态分配内存存储空间生命期：如果不显式地释放将一直保留到程序结束。

14. new 和 delete运算符

delete 运算符

✓用于释放堆内存。

✓两种格式：

delete 指针； //释放变量

delete[] 指针； //释放数组

✓不允许多次对同一块空间进行多次释放！delete只能与new配对使用，不能用它去释放不该释放的内存空间。

```
int *pInt = new int;
int *pManyint = new int[10]
delete  pInt;
delete[] pManyint;
```

```
int *pInt = new int[1];
delete pInt;
delete [] pInt;
```

?

14. new 和 delete运算符

```
int main()
{
  int arraysize;   //元素个数
  int *array;
  cout <<"please input a number of array:\n";
  cin >>arraysize;
  if((array=new int[arraysize])==NULL)  //分配堆内存
    {  cout <<"can't allocate more memory, terminating.\n";
       exit(0);
    }
  for(int count=0; count<arraysize; count++)
     array[count]=count*2;
  for(int count=0; count<arraysize; count++)
     cout <<array[count] <<"  ";
  cout <<endl;
  delete[] array;      //释放堆内存
  array=NULL;
}
```

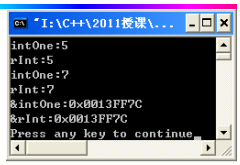
15. 引用

```
int main()
{
  int intOne;
  int & rInt=intOne;

  intOne=5;
  cout <<"intOne:" <<intOne <<endl;
  cout <<"rInt:" <<rInt <<endl;

  rInt=7;
  cout <<"intOne:" <<intOne <<endl;
  cout <<"rInt:" <<rInt <<endl;

  cout <<"&intOne:" <<&intOne <<endl;
  cout <<"&rInt:" <<&rInt <<endl;
}
```



✓对引用的操作就是对被引用的变量进行操作（对引用的改动，实际上就是对目标的改动）。

✓引用的地址就是目标的地址

15. 引用

✓引用就是为某个变量或隐含的临时变量起个别名，对别名的操作等同于对目标变量的操作。引用是C++所独有的。格式：

<数据类型> & <引用名 >= <同数据类型的变量>；

int a;

int & ia=a;

✓**规则：**

1、引用被创建的同时必须被初始化（指针可以在任何时候被初始化）。

2、不能有NULL的引用，引用必须与合法的存储单元关联（指针可以是NULL）。

3、一旦引用被初始化，就不能改变引用的关系（指针可以随时改变所指对象）。

15. 引用

✓**说明**

1、引用本身不是变量，它是某个对象的别名，其本身不占存储空间。

2、定义引用时必须指出目标对象，目标对象必须是单个对象且已定义或已声明。

✓**引用的特点为：**

1、引用声明后，对引用的操作等同于对目标对象的操作。

2、引用一旦初始化，它就维系在一个固定的目标上，再也不分开。

```
void main()
{
  int sum, avg, *p;
  int& rsum=sum;
  p=&sum;
  p=&avg;
}
```

rsum是变量sum的引用，类型为int&

15. 引用

对引用的操作

✓可以把某个引用的地址值赋给一个指针，而指针则指向被引用的变量。

✓可以把某个引用赋给变量，该变量的值便是被引用的变量值。

例：
```
int a = 3;
int & m = a;
m = m+5;  //实质上是a+5=8
int n = m;   //这时n=8
int *p = &m; //这是将指针p指向a变量的地址。
cout<<*p<<endl;
```

15. 引用

用引用传递函数参数

✓引用作为一般变量使用几乎没有什么实际意义，其最大的用处是用作函数的形参。

✓传递引用给函数与传递指针的效果一样，传递的是原来的变量或对象，而不是在函数作用域内建立变量或对象副本。

✓指针传递参数的缺点：在函数内需要重复使用*p；函数调用时需要传递变量的地址。

✓C++引入了引用使得函数在参数传递上显示出更强的威力。

15. 引用

例：

```
void swap(int &x,int &y) //引用作为函数形参
{ int temp;
  temp=x;   x=y;   y=temp;
}
void main( )
{ int a=5,b=6;
  cout<<a<<" "<<b<<endl;
  swap(a,b); //形参为引用，直接使用变量名作为函数实参
  cout<<a<<" "<<b<<endl;}
```

改成void swap(int x,int y)后结果如何？

程序运行结果是：
5 6
6 5

✓由于引用是“别名”，实参与形参之间不是值传递，而是一种“映射”，所以对形参的改变，实际上就是对实参的改变。

15. 引用

例：用指针实现相同的功能

```
void swap(int *x,int *y) //指针作为函数形参
{ int temp;
  temp=*x;   *x=*y;   *y=temp;
}
void main( )
{
  int a=5,b=6;
  cout<<a<<" "<<b<<endl;
  swap(&a,&b); //形参为指针，需地址名作为函数实参
  cout<<a<<" "<<b<<endl;}
```

swap(a,b); //引用
swap(&a,&b); //指针

✓由于通过指针的间接引用可以访问指针指向的变量的值，所以在被调函数中，通过指针间接引用改变了a、b的值

15. 引用

用引用返回值

```
float temp; //声明全局变量
float fn1(float r)
  { temp=r*r*3.14;
    return temp;  } //返回的是temp的值
float & fn2(float r)
  { temp=r*r*3.14;
    return temp;  } //返回的是temp的引用
int main( )
{ float a=fn1(5.0);     //一般函数返回方式
  float & b=fn1(5.0); //错,函数fn1返回类型是float，不是float&
  float c=fn2(5.0);   //通过temp的引用将temp的值赋给c
  float & d=fn2(5.0); //给全局temp起一个别名d
  cout<<a<<" "<<c<<" "<<d<<endl;}
```

程序运行结果是：78.5 78.5 78.5

15. 引用

不要返回局部变量的引用

如果把变量temp设置成局部的：

```
float & fn2(float r)
{
  float temp; //局部变量
  temp=r*r*3.14;
  return temp; //注意：返回一个局部变量的引用
}

int main( )
{ float &d=fn2(5.0);  //危险：d引用到一个局部变量上
  cout<<a<<" "<<c<<" "<<d<<endl;
}
```

int a = 10;
int & ra = a;
int & rra = ra; ?

END